Shielded Payments for Tangle Blueprints: Anonymous Pay-Per-Use Cloud Services via Cross-Chain Shielded Pools and Prepaid Credits

Drew Stone
drew@tangle.tools
Tangle Network

March 2026

Abstract

We present a privacy-preserving payment system for Tangle's Blueprint service protocol that enables anonymous pay-per-use access to cloud services including LLM inference, compute, and storage. The system composes three layers: (1) a cross-chain variable-amount shielded pool based on the Webb Anchor System [1] supporting multi-asset wrapping, (2) a gateway contract that bridges anonymous pool withdrawals to Tangle's service lifecycle, and (3) a prepaid credit scheme where a single on-chain zero-knowledge proof funds an account from which many jobs are authorized via cheap EIP-712 signatures. The construction requires no modifications to Tangle's audited core contracts and introduces only 664 lines of new Solidity as auditable surface. We provide formal definitions, security properties, and a complete implementation with 30 passing tests including end-to-end flows.

1 Introduction

Decentralized service marketplaces face a fundamental tension: users want privacy when consuming services, but operators need payment guarantees before performing work. Existing systems force users to reveal their identity with every payment, creating surveillance vectors that centralized alternatives avoid through corporate privacy policies rather than cryptographic guarantees.

Buterin and Crapis [2] proposed "ZK API Usage Credits" for anonymous LLM inference: users deposit into a smart contract and generate zero-knowledge proofs to authorize each API call without revealing their identity. While elegant, generating a zkSNARK proof per API call is impractical—proof generation takes ∼10 seconds and costs ∼300k gas on-chain.

We extend this idea with a two-phase design:

1. **Funding phase**: One zkSNARK proof (VAnchor withdrawal) creates a pseudonymous credit account.
2. **Spending phase**: Many ECDSA signatures (∼3k gas each) authorize individual job payments.

This paper describes the complete system built on Tangle's existing Blueprint protocol, reusing the audited Webb Protocol shielded pool [1] for the funding phase and introducing a minimal credit contract for the spending phase.

1.1 Contributions

- A gateway architecture that bridges VAnchor shielded pools to Tangle's service lifecycle with zero modifications to the core protocol.

- A prepaid credit scheme using EIP-712 typed signatures that amortizes the cost of a single ZK proof over arbitrarily many service invocations.
- Multi-asset wrapping via `FungibleTokenWrapper`, allowing users to deposit any supported stablecoin into a single anonymity pool.
- An implementation with 664 lines of new Solidity, 30 passing Forge tests, and a TypeScript SDK.

2 Preliminaries

We adopt the notation from the Webb Protocol [1]. Let $H : \mathbb{F}_p^* \to \mathbb{F}_p$ denote a collision-resistant hash function (Poseidon). Let $(\mathsf{pk}, \mathsf{sk})$ denote a keypair where $\mathsf{pk} = H(\mathsf{sk})$.

Definition 1 (UTXO Commitment)**.** A commitment to a UTXO with chain identifier i, value v, public key pk, and blinding factor $r \xleftarrow{\$} \mathbb{F}_p$ is:

$$\mathsf{cm} = H(i, v, \mathsf{pk}, r)$$

Definition 2 (Nullifier)**.** The nullifier for a commitment cm at Merkle path index j with signature $\sigma = H(\mathsf{sk}, j, \mathsf{cm})$ is:

$$\mathsf{sn} = H(\mathsf{cm}, j, \sigma)$$

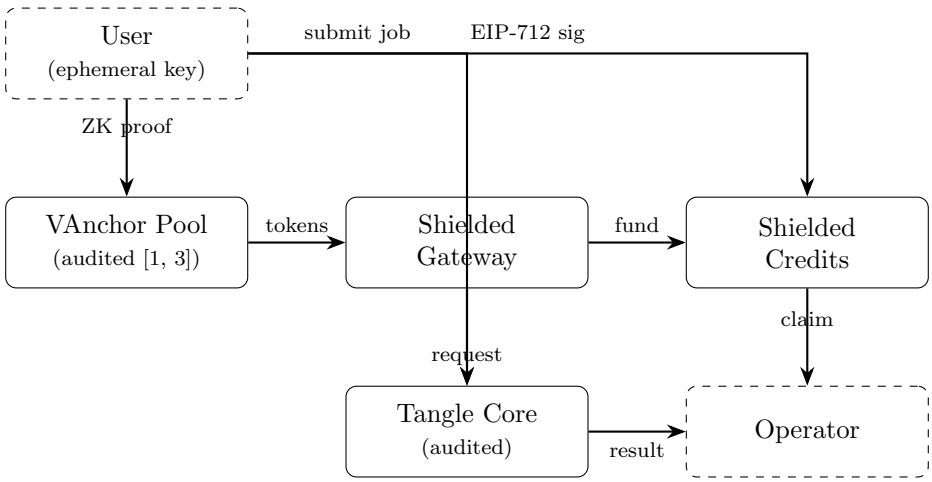
Definition 3 (Cross-Chain Merkle Membership)**.** Given a set of Merkle roots $\mathbf{MR} = \{\mathsf{MR}_1, \ldots, \mathsf{MR}_k\}$ from k connected anchor instances, a commitment cm satisfies cross-chain membership if:

$$\mathsf{MR}_{\mathsf{cm}} := \mathsf{reconstruct}(\mathsf{cm}, \mathsf{path}(\mathsf{cm}), H) \in \mathbf{MR}$$

Definition 4 (Tangle Service)**.** A *service* is a running instance of a *blueprint* (service template) with assigned operators. Services support three pricing models: PayOnce (upfront), Subscription (periodic escrow), and EventDriven (per-job). Users interact via *permitted callers* authorized at service creation.

3 System Architecture

The system comprises three contracts and one SDK:



3.1 VAnchor Shielded Pool (Existing, Audited)

We reuse the Variable Asset Anchor System from the Webb Protocol [1] without modification. The VAnchor implements a UTXO-based shielded pool with:

- **Variable amounts**: Unlike fixed-denomination mixers, UTXOs carry arbitrary values.
- **JoinSplit transactions**: 2-in-2-out or 16-in-2-out for splits, merges, and transfers.
- **Cross-chain roots**: `ManyMerkleProof` validates a commitment against roots from any connected chain.
- **Multi-asset wrapping**: `FungibleTokenWrapper` accepts multiple ERC20s (e.g., USDC, USDT, DAI) into a single wrapped token, maximizing the anonymity set.

The zkSNARK relation for VAnchor transactions (from [1], Example 2):

Construction 1 (VAnchor Transaction Gadget)**.** For public inputs $x = \{\mathsf{public_amount}, \mathbf{MR}, i, \{\mathsf{sn}_k\}_{k \le \mathsf{INS}}, \{\mathsf{cm}$ and private witness $\mathbf{w} = \{v_k, r_k, \mathsf{sk}_k, j_k, \mathsf{path}(\mathsf{cm}_k)\}_{k \le \mathsf{INS}} \cup \{v_o, r_o, i_o, \mathsf{sk}_o, j_o\}_{o \le \mathsf{OUTS}}$, the relation $\mathcal{R}$ enforces:

$$\{\mathsf{pk}_k := H(\mathsf{sk}_k)\}_{k \le \mathsf{INS}} \tag{1}$$

$$\{\mathsf{cm}_k := H(i, v_k, \mathsf{pk}_k, r_k)\}_{k \le \mathsf{INS}} \tag{2}$$

$$\{\sigma_k := H(\mathsf{sk}_k, j_k, \mathsf{cm}_k)\}_{k \le \mathsf{INS}} \tag{3}$$

$$\{\mathsf{MR}_{\mathsf{cm}_k} := \mathsf{reconstruct}(\mathsf{cm}_k, \mathsf{path}(\mathsf{cm}_k), H) \in \mathbf{MR}\}_{k \le \mathsf{INS}} \tag{4}$$

$$\mathsf{public_amount} + \sum_{k \le \mathsf{INS}} v_k = \sum_{o \le \mathsf{OUTS}} v_o \tag{5}$$

$$\{\mathsf{sn}_k\}_{k \le \mathsf{INS}} \text{ are distinct} \tag{6}$$

3.2 Shielded Gateway

The `ShieldedGateway` contract is a stateless proxy between the VAnchor and Tangle. It executes a VAnchor withdrawal (which validates the zkSNARK proof and marks nullifiers as spent), receives the withdrawn tokens, and atomically forwards them to one of three Tangle operations:

1. `shieldedRequestService`: Creates a new service with the gateway as owner and user-specified ephemeral addresses as permitted callers.
2. `shieldedFundService`: Tops up an existing service's escrow.
3. `shieldedFundCredits`: Funds a prepaid credit account (Section 3.3).

Property 1 (Atomicity)**.** The gateway never holds funds beyond a single transaction. Every token received from the VAnchor is forwarded in the same call. If the forward fails, the entire transaction reverts (including the VAnchor withdrawal).

Property 2 (Non-Linkability)**.** The gateway stores no mapping between VAnchor nullifiers and Tangle service/credit identifiers. An observer sees `VAnchor` → `Gateway` → `Tangle/Credits` but cannot determine which pool depositor initiated the operation.

3.3 Shielded Credits

The core contribution: a prepaid credit system that amortizes a single ZK proof over many service invocations.

Definition 5 (Credit Account)**.** A credit account is identified by a commitment $\mathsf{C} = \mathsf{keccak256}(\mathsf{pk}_s \| \mathsf{salt})$ where pk_s is an ephemeral ECDSA public key and $\mathsf{salt} \xleftarrow{\$} \{0,1\}^{256}$. The account state is:

$$\mathsf{Account}[\mathsf{C}] = (\mathsf{pk}_s, \mathsf{token}, \mathsf{balance}, \mathsf{nonce})$$

Definition 6 (Spend Authorization)**.** A spend authorization for credit account C is an EIP-712 typed data signature:

$$\mathsf{SpendAuth} = \mathsf{Sign}_{\mathsf{sk}_s}\Big(\mathsf{C}, \mathsf{serviceId}, \mathsf{jobIndex}, \mathsf{amount}, \mathsf{operator}, \mathsf{nonce}, \mathsf{expiry}\Big)$$

where sk_s is the ephemeral private key corresponding to pk_s.

3.3.1 Credit Protocol

Algorithm 1 Credit Lifecycle

```
procedure FundCredits(token, amount, C, pk_s)
    Pull amount of token from caller (Gateway)
    if Account[C] exists then
        require Account[C].pk_s = pk_s                    ▷ Immutable key
    else
        Account[C] ← (pk_s, token, 0, 0)
    end if
    Account[C].balance += amount
end procedure

procedure AuthorizeSpend(auth)
    acct ← Account[auth.C]
    require auth.nonce = acct.nonce                       ▷ Strict ordering
    require auth.amount ≤ acct.balance
    require block.timestamp ≤ auth.expiry
    require ecrecover(digest(auth)) = acct.pk_s
    acct.balance −= auth.amount;   acct.nonce + +
    authHash ← H(auth.C, auth.serviceId, auth.jobIndex, auth.nonce)
    Store Pending[authHash] ← (auth.amount, auth.operator)
    return authHash
end procedure

procedure ClaimPayment(authHash, recipient)
    require msg.sender = Pending[authHash].operator
    require not Claimed[authHash]
    Claimed[authHash] ← true
    Transfer Pending[authHash].amount to recipient
end procedure
```

4 Security Analysis

4.1 Anonymity

Property 3 (Depositor Unlinkability)**.** The anonymity set for a credit account C is the set of all depositors into the VAnchor pool for the same wrapped token. Neither the gateway nor the credits contract stores the depositor's address. The on-chain trace reveals only:

$$\texttt{VAnchor} \xrightarrow{\mathsf{tokens}} \texttt{Gateway} \xrightarrow{\mathsf{tokens}} \texttt{Credits}[\mathsf{C}]$$

The commitment C is derived from a freshly generated ephemeral key and cannot be linked to the depositor.

Property 4 (Spend Pseudonymity)**.** Spend authorizations reveal the commitment C and the designated operator. Multiple spends from the same C are linkable to each other (they share a pseudonym) but not to the original depositor. This is a deliberate tradeoff: full unlinkability per-spend would require a ZK proof per job, which we avoid for efficiency.

4.2 Payment Security

Property 5 (No Double-Spend)**.** Credit balance is deducted atomically in AuthorizeSpend before the authorization is stored. The strict nonce ordering ($\mathsf{auth.nonce} = \mathsf{acct.nonce}$) prevents replay and out-of-order execution.

Property 6 (No Front-Running Claims)**.** Each spend authorization designates an operator address. Only $\mathsf{msg.sender} = \mathsf{auth.operator}$ can call ClaimPayment. An observer who sees the `CreditsSpent` event cannot claim the payment.

Property 7 (Spending Key Immutability)**.** Once a credit account is funded with spending key pk_s, subsequent top-ups must use the same key. This prevents an attacker who discovers C from re-keying the account during a race condition.

Property 8 (Expiry Bounds)**.** Spend authorizations include an expiry timestamp. This limits the window for griefing attacks where an authorized spend is delayed indefinitely.

4.3 Gateway Security

Property 9 (Balance Verification)**.** The gateway records its token balance before calling `VAnchor.transact()` and verifies the balance increased by at least the expected withdrawal amount afterward. This prevents the VAnchor from transferring less than claimed.

Property 10 (Nullifier Enforcement)**.** Double-spend prevention is inherited from the VAnchor: each UTXO nullifier can only be marked as spent once. The gateway does not implement its own nullifier tracking.

5 Anonymous Inference: End-to-End Flow

We describe the complete user journey for anonymous LLM inference, the motivating use case.

5.1 Setup (One-Time)

1. **Operator registers**: An operator registers a blueprint describing an inference service (e.g., vLLM with a specific model), stakes collateral via Tangle's `MultiAssetDelegation`, and runs a service node.
2. **Pool deployment**: A `FungibleTokenWrapper` is deployed accepting USDC, USDT, and DAI. A `VAnchorTree` is deployed with this wrapper. The `ShieldedGateway` and `ShieldedCredits` contracts are deployed and the pool is registered.

5.2 User Flow

1. **Deposit** (public, one-time): User wraps 100 USDC into the VAnchor pool via `FungibleTokenWrapper.wr`
   This is visible on-chain but joins a large anonymity set.
2. **Generate keys** (off-chain): User generates an ephemeral ECDSA keypair $(\mathsf{sk}_s, \mathsf{pk}_s)$ and computes $\mathsf{C} = \mathsf{keccak256}(\mathsf{pk}_s \| \mathsf{salt})$.
3. **Fund credits** (one ZK proof): User generates a VAnchor withdrawal proof and calls:

$$\texttt{gateway.shieldedFundCredits}(\pi_{\mathsf{VAnchor}}, \mathsf{C}, \mathsf{pk}_s)$$

   This is the *only* ZK proof in the entire flow. Cost: ∼10s off-chain, ∼300k gas on-chain.

4. **Create service** (optional, one ZK proof): If no service exists, user calls:

$$\texttt{gateway.shieldedRequestService}(\pi_{\mathsf{VAnchor}}, \mathsf{params})$$

with pk_s as a permitted caller. Alternatively, the user joins an existing service.
5. **Submit jobs** (repeated, cheap): For each inference request:
   (a) User signs: $\mathsf{auth} \leftarrow \mathsf{Sign}_{\mathsf{sk}_s}(\mathsf{C}, \mathsf{serviceId}, \mathsf{jobIndex}, \mathsf{price}, \mathsf{operator}, \mathsf{nonce}, \mathsf{expiry})$
   (b) Submit auth on-chain: `credits.authorizeSpend(auth)` (∼50k gas)
   (c) Submit job to Tangle: `tangle.submitJob(serviceId, jobIndex, prompt)` using pk_s (∼80k gas)
6. **Operator claims**: After serving the request, the operator calls:

$$\texttt{credits.claimPayment}(\mathsf{authHash}, \mathsf{operatorAddr})$$

7. **Withdraw remainder** (when done): User signs a withdrawal and reclaims unused credits.

5.3 Cost Comparison

Approach	Per-job proof	Per-job gas	Per-job latency
Buterin/Crapis (ZK per call)	Groth16	∼300k	∼10s
Direct VAnchor withdrawal	Groth16	∼300k	∼10s
Shielded Credits (ours)	**ECDSA sig**	**∼50k**	**<100ms**
Plain x402 (no privacy)	None	∼30k	<50ms

6 Audit Surface Analysis

A key design goal is minimizing new code that requires auditing.

Component	Status	Lines	Source
VAnchor + TokenWrapper + Merkle	Audited (Veridise [3])	∼3000	Git submodule
Tangle core (Payments, Services, Jobs)	Audited	∼8000	Existing
OpenZeppelin 4.x / 5.x	Audited	—	Dependencies
`ShieldedGateway`	**New**	196	`src/shielded/`
`ShieldedCredits`	**New**	216	`src/shielded/`
Interfaces (`IShielded*`, `IVAnchor`)	**New**	252	`src/shielded/`
Total new Solidity		**664**	

7 Implementation

The implementation is available on branch `feat/shielded-payments` of the `tnt-core` repository.

7.1 Solidity Contracts

- `ShieldedGateway.sol`: Stateless proxy. Executes VAnchor withdrawals and forwards to Tangle or Credits. Inherits `Ownable` (pool registration) and `ReentrancyGuard`.
- `ShieldedCredits.sol`: Credit accounts with EIP-712 spend authorizations. Strict nonce ordering, designated operator claims, spending key immutability.
- `IVAnchor.sol`: Typed interface for `VAnchorTree.transact()` to avoid low-level calls.

7.2 TypeScript SDK

The `@tangle-network/shielded-sdk` package provides:

- **Protocol**: Keypair, UTXO, Poseidon hash, Merkle tree, encrypted outputs.
- **Proof**: Witness builder, Groth16 prover (via snarkjs), Solidity proof encoder, cross-chain root encoding, circuit artifact download/caching.
- **Notes**: Serialization, FIFO selection, file/memory storage backends.
- **Contracts**: `ShieldedGatewayClient`, `ShieldedCreditsClient`, on-chain leaf sync, note discovery.
- **Deploy**: Full-stack deployment script (Poseidon bytecode injection, verifiers, wrapper, VAnchor, gateway, credits).

7.3 Test Coverage

30 Forge tests across three suites:

- **ShieldedGatewayTest** (10 tests): Pool registration, shielded service requests, shielded escrow funding, nullifier replay, recipient validation, fund retention invariant.
- **ShieldedCreditsTest** (17 tests): Funding, top-up, key immutability, token mismatch, signature verification, nonce replay, insufficient balance, expiry, sequential spends, operator-gated claims, double-claim prevention, withdrawal.
- **ShieldedE2ETest** (3 tests): Full anonymous inference flow (fund $\rightarrow$ create service $\rightarrow$ 5 jobs $\rightarrow$ claim $\rightarrow$ withdraw), front-running prevention, credits/escrow independence.

8 Discussion

8.1 Tradeoffs

Pseudonymity vs. full unlinkability. Our design reveals the credit commitment C on each spend, allowing an observer to link multiple jobs to the same pseudonym. Full per-job unlinkability would require a ZK proof per job (the Buterin/Crapis approach), which we reject for performance. For most use cases—inference, compute, storage—pseudonymous access is sufficient.

Trust in operators. The credit system pays operators upon claim, not upon verified job completion. While Tangle's staking and slashing mechanisms provide economic security against operator misbehavior, the credits contract itself does not verify job completion on-chain. Future work could integrate with Tangle's result verification hooks.

Cross-chain privacy. The VAnchor's cross-chain root relay allows deposits on chain A and spending on chain B. However, the credit system and Tangle services live on a single chain. Cross-chain credits would require additional bridge infrastructure.

8.2 Future Work

- Integration with rate-limiting nullifiers (RLN) for Sybil resistance without identity.
- TEE-based operator attestation for execution confidentiality (complementing payment privacy).
- Cross-chain credit portability via the existing `SignatureBridge` mechanism.
- Streaming payment channels for sub-second billing granularity.

References

[1] D. Stone, "Webb Protocol: A cross-chain private application and governance protocol," *IACR ePrint 2023/260*, 2023.

[2] V. Buterin and D. Crapis, "ZK API Usage Credits," presented at ETHMumbai, March 2026.

[3] Veridise Inc., "Security audit of Webb Protocol Solidity contracts," 2023. Available in `protocol-solidity` repository.

[4] A. Pertsev, R. Semenov, and R. Storm, "Tornado Cash privacy solution," 2019.

[5] R. Floersch and B. Johnson, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, 2017.

[6] L. Grassi *et al.*, "Poseidon: A new hash function for zero-knowledge proof systems," *USENIX Security*, 2021.

[7] J. Groth, "On the size of pairing-based non-interactive arguments," *EUROCRYPT*, 2016.